Hackathon Specification

Kombee AI Engineering Team Hackathon

Prepared By

Kombee Technologies (I) Pvt. Ltd.

517-520, Homeland City,

Vesu, Surat, Gujarat - 395007

Index

1. Objective....4
2. Hackathon Structure....4
3. Phase 1 - Build Kombee AI Backbone Layer....4
4. Phase 2 - Practical Demonstration....7
5. Deliverables....10
6. Evaluation Criteria (100 Points)....11
7. Final Goal....11

Building Kombee's AI Backbone

⏱ Duration: **8 Hours**

Participants:

AI Engineers / AI Lab / AI Department

1. Objective

As Kombee develops multiple AI-powered utilities across different departments, it is essential to build a **central AI orchestration layer** that ensures:

- Correct model selection
- Cost optimization
- Response validation
- Observability
- Reusability across products

The goal of this hackathon is to **design and implement the foundational AI backbone layer** that future Kombee AI applications can use.

Participants must also **demonstrate their working through a practical AI application**.

2. Hackathon Structure

The challenge is divided into **two phases**.

Phases	Focus
Phase 1	Build the **AI Backbone Orchestration Layer**
Phase 2	Demonstrate its usage with a **Practical AI Application**

3. Phase 1 - Build Kombee AI Backbone Layer

Participants must design and implement a **central AI orchestration layer** that will act as the **core backbone logic layer** for all future AI utilities developed at Kombee.

This backbone should enable:

- Multi-model orchestration
- Cost control

- AI validation
- Response reliability

- **Reference Framework**
  - Participants must follow the model selection and benchmarking approach described in the internal documentation.

Reference pages:

- LLM Selection Framework

  https://kombee-technologies.atlassian.net/wiki/spaces/AD1/pages/4433641480/LLM+Selection+Framework+How+to+Identify+the+Right+Model+for+Your+Use+Case
- LLM Comparison Platforms

  https://kombee-technologies.atlassian.net/wiki/spaces/AD1/pages/4433313794/LLM+Comparison+Platforms+Evaluating+Models+for+Specific+Use+Cases

- **Expected Backbone Architecture**

  Participants should implement the following architecture.

```
User / Internal System
        |
        ▼
API Gateway
        |
        ▼
Use Case Classification Layer
        |
        ▼
AI Orchestrator Core
        |
├── Task Complexity Engine
├── Cost Policy Engine
├── Risk & Compliance Validator
├── Context Builder
├── Model Router
        |
```

```
        ▼
Multi-Model Execution Layer
        |
├── Reasoning Model
├── Coding Model
├── Lightweight Model
        |
        ▼
Validation & Guardrail Layer
        |
├── JSON Validator
├── Code Lint / Compile Check
├── Hallucination Detector
├── Tool Call Validator
        |
        ▼
Self-Healing Loop (Retry ≤ 2)
        |
        ▼
Response Formatter
        |
        ▼
Observability + Cost Logging
        |
        ▼
Final Output
```

- **Required Technologies**

Participants must use the following components:

- LangChain
- Retrieval-Augmented Generation (RAG)

Any free or open-source vector database such as:

- Chroma
- FAISS

- Weaviate

LLM Usage Constraint

Since only **free API keys are allowed**, the implementation should use:

- Google Gemini
- However, participants must include **commented code showing how other models would be used**, such as:
- Reasoning model
- Coding model
- Lightweight model

4. Phase 2 - Practical Demonstration

- Participants must demonstrate the backbone layer using a **real-world AI application**.
- The example application will simulate a **restaurant inventory planning assistant** for the quick-commerce platform:
    - Zepto

- **Application Goal**
- Build a **RAG-based chatbot** that helps a restaurant plan ingredient purchase based on:
- Menu items
- Current stock
- Past purchase history
- Seasonal patterns

- **User Flow**
    - User uploads restaurant menu
    - AI extracts food items from the menu
    - AI identifies ingredients required to prepare those items
    - System checks ingredient availability in database
    - AI predicts required quantity for next purchase

Final output:

    - A **cart table of ingredients that should be ordered**.

- **Required Data Setup**
  - Participants must generate a **dummy database**.

Recommended database:

- PostgreSQL
- **Database Requirements**

Participants must create the following tables:

- **Users Table**

Minimum:

- 10 unique users

Products Table

Minimum:

50,000 unique products

- **Orders Table**

Minimum:

- 2,000 orders per user

Approx total:

- 20,000 orders

- **Order Details Table**

Each order should have:

- 5 to 15 product entries.

Database Setup Constraint

The entire database setup should take **no more than 30 minutes**.

Participants may use:

- SQL scripts
- Python scripts
- Data generation tools

- **RAG Knowledge Base**

- The chatbot must support **menu document ingestion**.

User uploads:

- Restaurant menu (PDF or text)

The system extracts:

- Dish names
- Ingredients
  - These must be stored in the **vector database** for retrieval.

- **Prediction Requirement**
  - Participants must design a **purchase quantity prediction logic** based on historical order data.

Prediction factors may include:

- Order frequency
- Time of day
- Date
- Season
- User purchasing behavior
  - The system should estimate **how much quantity should be ordered for each ingredient**.

- **Expected Output**
  - The chatbot response should generate a **structured table** showing:
  - | Ingredient | Available Stock | Predicted Required Quantity | Recommended Order Quantity |
  - Only ingredients that require ordering should appear in the output.

5. Deliverables

Participants must submit:

1. Source Code Repository

Includes:

- Backbone orchestration layer
- RAG chatbot implementation
- Database generation scripts

2. Database Setup Script

Script that generates dummy data quickly.

3. Architecture Documentation

Explain:

- Backbone architecture
- Model routing strategy
- Guardrails implementation
- RAG design

4. Recorded Demo Video

Participants must record a working demonstration showing:

- Backbone layer architecture
- Menu upload
- Ingredient extraction
- Database query
- Quantity prediction
- Final cart generation
- Commit the video file(s) at the **root of your GitHub repository**.

5. **GitHub Submission**
    a. Use your Kombee GitHub ID
    b. Push complete project
    c. Include video(s) at root
    d. Submit GitHub repository link to your respective department lead

6. **AI-Based Evaluation**
    a. We will use AI tools to evaluate:
    b. Your video explanation clarity
    c. Since AI will evaluate:
        i. Make your video extremely detailed
        ii. Explain reasoning clearly
        iii. Avoid skipping steps
        iv. Show real evidence
        v. Superficial explanations will score lower.

Deadline: **14th March, 8:00 PM**

6. Evaluation Criteria (100 Points)

Category	Points
AI Backbone Architecture	25
Orchestration & Model Routing	15
RAG Implementation	15
Prediction Logic	15
Database Design	10
Code Quality	10
Demo & Explanation	10

7. Final Goal

The objective of this challenge is not only to build a prototype but to **lay the foundation for Kombee's internal AI platform**.

The best solutions should demonstrate:

- Robust architecture
- Reusable AI infrastructure
- Practical real-world application